

Human Activity Recognition with a Hidden Markov Model

1. Background and Motivation

Human Activity Recognition (HAR) is important because it is used to figure out a person's physical activities without any human assistance. However, HAR faces a major problem which is that smartphones often capture messy data that is inconsistent and requires heavy computers to process[1]. In this project, the goal is to track fitness and activity by using smartphone accelerometer and gyroscope measurements to recognise walking, standing, jumping, and still when there is no movement. I used a Hidden Markov Model because it captures the activity as a sequence of hidden states while accounting for temporal dependencies between consecutive observations, allowing noisy sensor predictions to be smoothed into more consistent activity classifications over time. This temporal modeling makes HMMs well suited for improving the reliability of smartphone-based fitness monitoring systems, where activities naturally occur in continuous sequences rather than as isolated events [1], [2].

2. Data Collection and Preprocessing

2.1 Collection Protocol

I used the Sensor Logging app [3] in a single device with a fixed sampling rate of FS=100HZ to record 4 activities: standing, walking, jumping, and still. I recorded 13 trials per activity each between 7 to 10 sec totalling to 52 trials.

TABLE I
DATA COLLECTION SUMMARY

Activity	No. of trials	Total Duration (s)	Mean duration (s)
Jumping	13	113.80	8.75
Standing	13	106.81	8.22
Still	13	112.61	8.66

Walking	13	118.67	9.13
---------	----	--------	------

2.2 Raw signal visualization

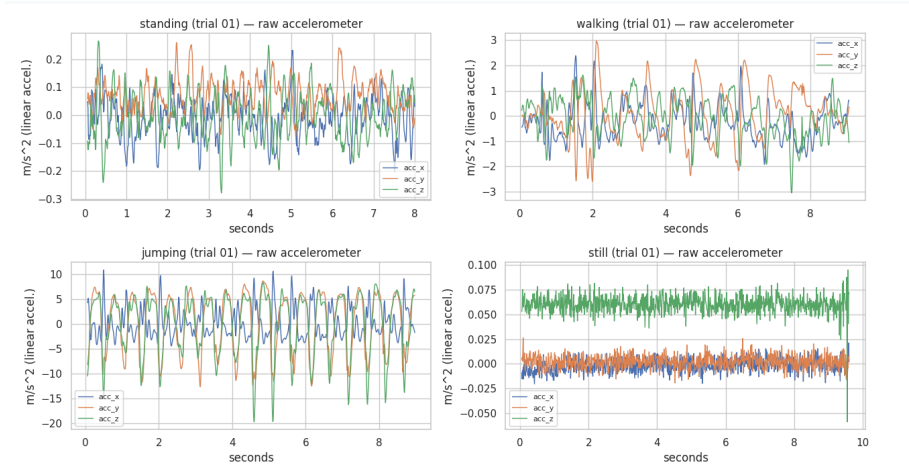


Fig. 1. Raw signals by activity

Jumping and walking show clear periodic oscillation in the accelerometer axes, standing shows small, irregular hand-tremor-scale movement, and still is visibly flat by comparison.

2.3 Windowing strategy

There was a 50% overlap caused by the 100 samples (1.0s) and the step size of 50 samples (0.5s). A second window at 100Hz reliably captures at least one full cycle of movement which is what makes the dominant-frequency and spectral-energy features meaningful. The purpose of the overlap is to allow more training windows per trial for Baum-Welch.

3. Feature Extraction (Time and Frequency Domain)

41 features per window were computed with zero NaNs across 832 total windows. Table II below shows the extracted features.

TABLE II
FEATURE SET

Domain	Feature	Applied to	Count
Time	Mean, std, min, max	acc_x/y/z, gyro_x/y/z (6 axes)	24
Frequency	Dominant frequency, spectral energy (FFT)	acc_x/y/z, gyro_x/y/z (6 axes)	12
Time	Signal magnitude area (SMA)	Accel, gyro	2
Time	Cross-axis correlation	acc_xy, acc_xz, acc_yz	3
Total			41

To prevent data leakage during model evaluation, the scaler was fitted only on the training data to ensure information from the test set did not affect feature scaling[4].

TABLE III
WINDOWED DATASET COMPOSITION

Activity	No. of Feature windows
walking	221
jumping	209
still	206
standing	196
Total	832

4. HMM Setup and Implementation

4.1 Model components

These are the standard components I used for this study.

TABLE IV
HIDDEN MARKOV MODEL COMPONENTS

Element	Definition in this project
Hidden states (Z)	I used 4 hidden states : jumping, walking, still, and standing.
Observations (x)	A 41-dimensional standardized feature vector was extracted from each 1-second window .
Transition probabilities (A)	I estimated the Baum-Welch algorithm using the 4×4 transition probability matrix and the initial state probability distribution .
Emission probabilities (B)	The diagonal-covariance Gaussian distribution was used to model the emission probabilities.
Initial state probabilities (π)	The initial probability distribution learned automatically during Baum–Welch training .

4.2 Train/test split

Instead of splitting individual windows, the dataset was split by whole trial to prevent data leakage. This ensured overlapping windows from the same recording did not appear in both training and test sets. This resulted in a training set of 632 windows from 40 trials and a test set of 200 windows from 12 unseen trials. This shows the ability of the Hidden Markov Model to generalize new recordings.

4.3 Model fitting (Baum-Welch)

The model converged successfully with 7 iterations because it was configured with a diagonal covariance of `n_iter=200`, `tol=1e-3`. Training stopped automatically when the model converged because of the `model.monitor_.converged`.

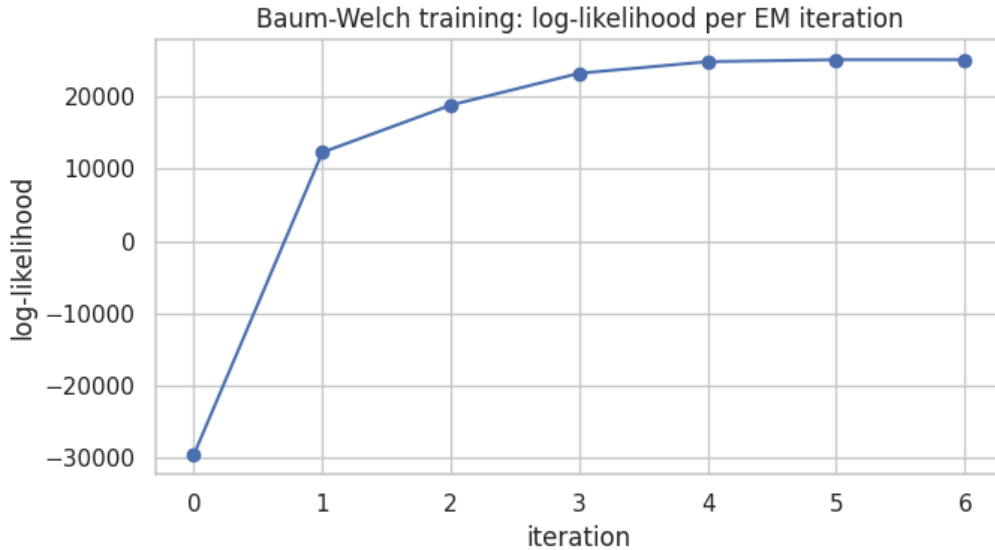


Fig. 2. Log-likelihood versus Baum–Welch (EM) iteration during Hidden Markov Model training.

According to figure 2 above, the curve rises gradually then flattens indicating that the Hidden Markov Model was trained using the Baum-Welch EM algorithm [5] and the sequence was inferred using the Viterbi algorithm [6] which separates the classes.

4.4 State-to-label mapping

All 4 states were mapped to a different activity ensuring that no two states collapse to the same label.

4.5 Transition matrix

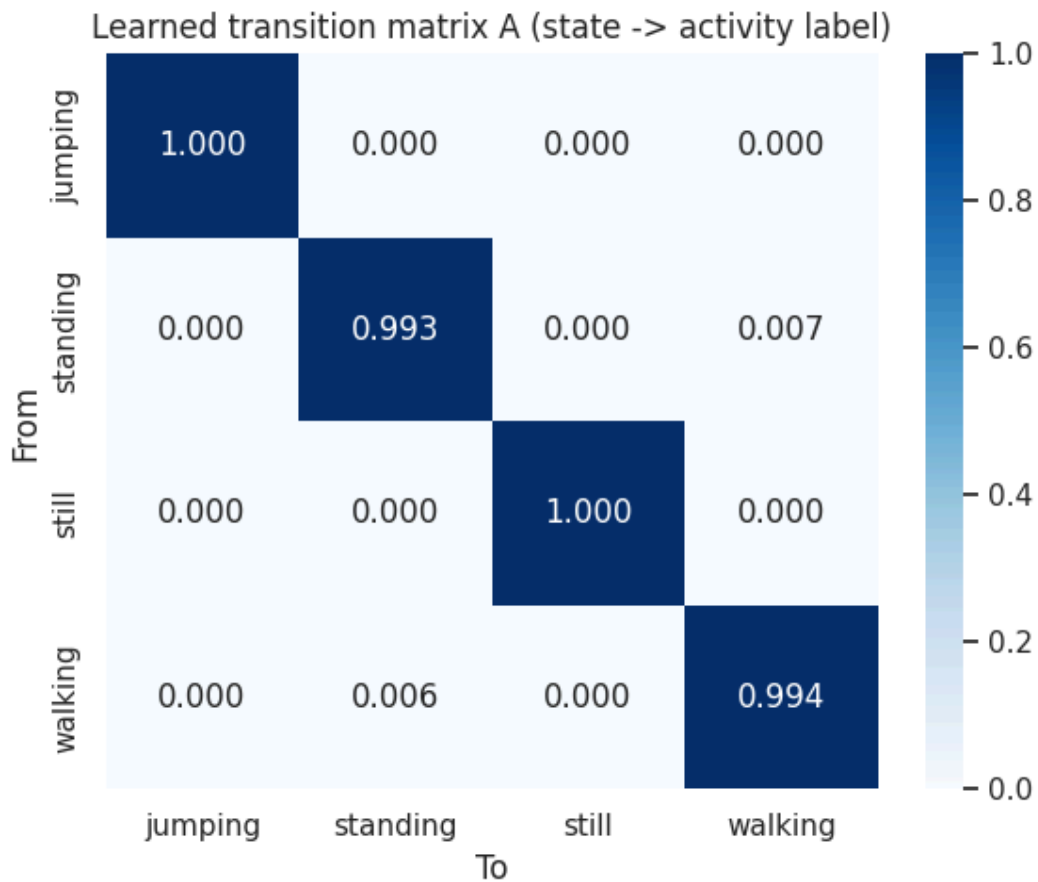


Fig. 3. Transition Matric Heatmap

The transition matrix is strongly diagonal because each training is individual; there is no mixing therefore Baum-Welch never observes a real activity transitioning to another. This happened because I recorded each activity individually. If I were to record sequentially by walking, then jumping, then standing, and still, Baum-Welch would have observed an activity transitioning to another.

4.6 Initial-state probabilities

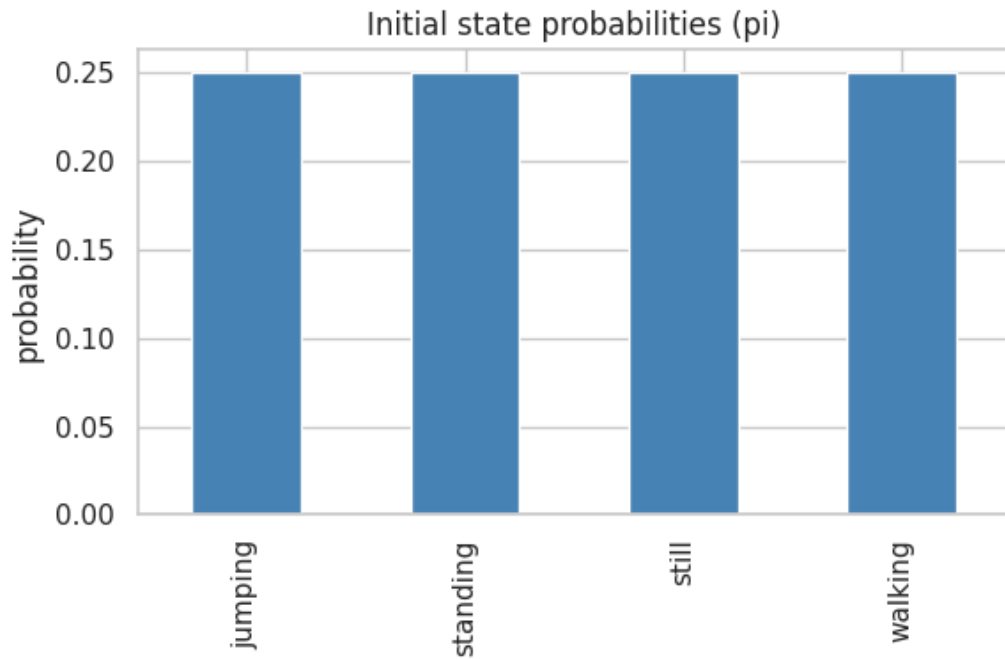


Fig. 4. Initial state probabilities

According to figure 4, the initial state probabilities of all 4 states are 0.25. This is because the training set is balanced therefore the Baum-Welch algorithm found no strong evidence that one hidden state should be the starting state of another.

4.7 Emission probabilities

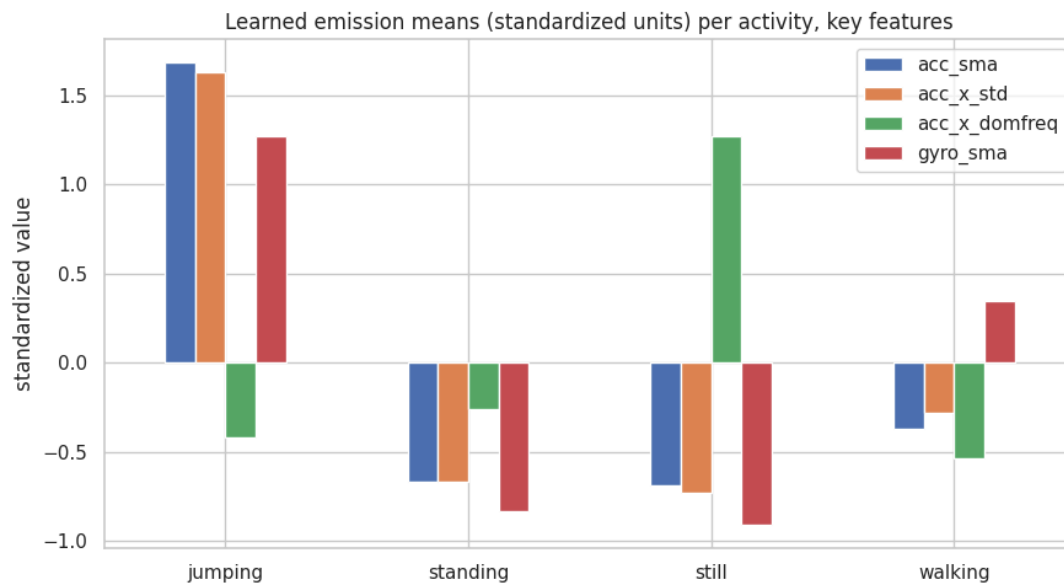


Fig. 5. Emission means key features.

TABLE V
EMISSION MEANS, KEY FEATURES

Activity	acc_sma	acc_x_std	acc_x_domfreq	gyro_sma
jumping	1.684	1.628	-0.418	1.274
standing	-0.666	-0.008	-0.263	-0.833
still	-0.689	-0.728	1.269	-0.908
Walking	-0.371	-0.282	-0.534	0.348

Jumping has the highest SMA/std because of high energy movement while still has the lowest.

5. Results and Evaluation on Unseen Data

I recorded 13 trials for each activity. Trials 1-10 were used for training the model while trials 11-13 were used to test the performance of the model. Therefore, the dataset was split by a whole trial to prevent data leakage. This ensured overlapping windows from the same recording did not appear in both training and test sets. This resulted in a training set of 632 windows from 40 trials and a test set of 200 windows from 12 unseen trials. This shows the ability of the Hidden Markov Model to generalize new recordings.

5.1 Viterbi decoding

The Hidden Markov Model used the Viterbi algorithm to record the hidden activity sequence for each trial.

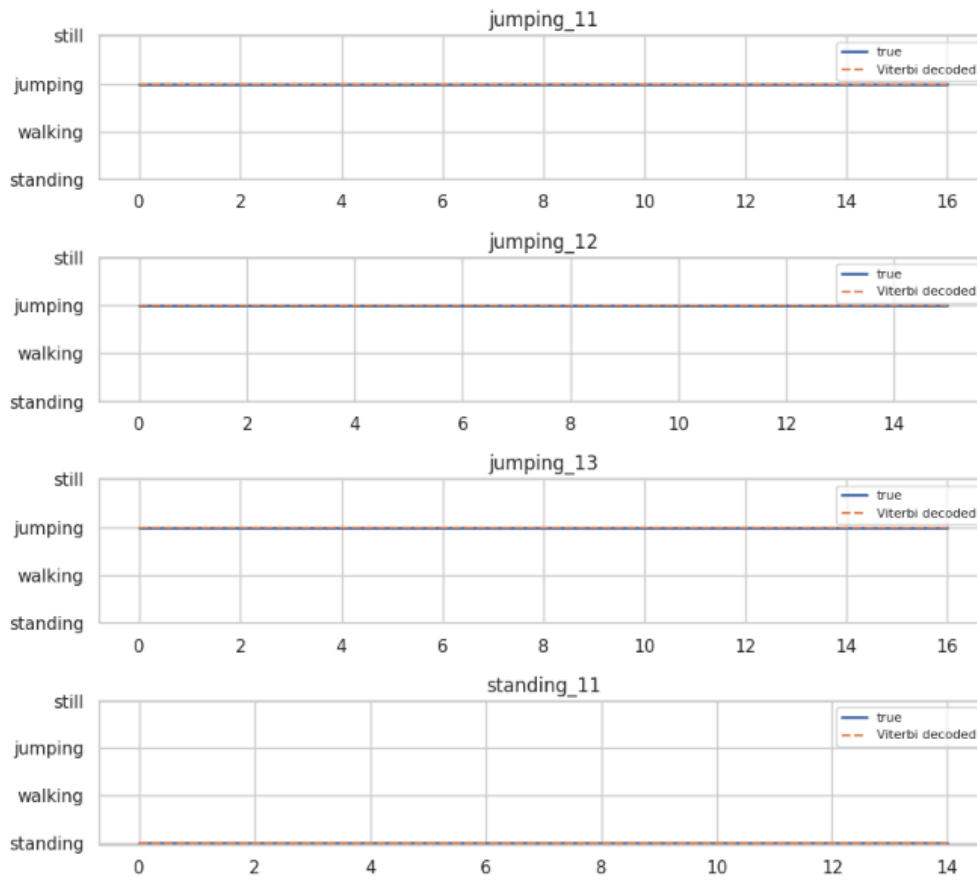


Fig. 6. Decoded vs true sequences

Figure 6 above shows the predictions are stable from one window to the next meaning that the HMM model uses temporal continuity to produce smooth predictions and reduce errors.

5.2 Confusion matrix

Figure 7 and Table VI show the confusion matrix results of the test split data. There was no misclassification between the 4 activities resulting in a perfectly diagonal confusion matrix.

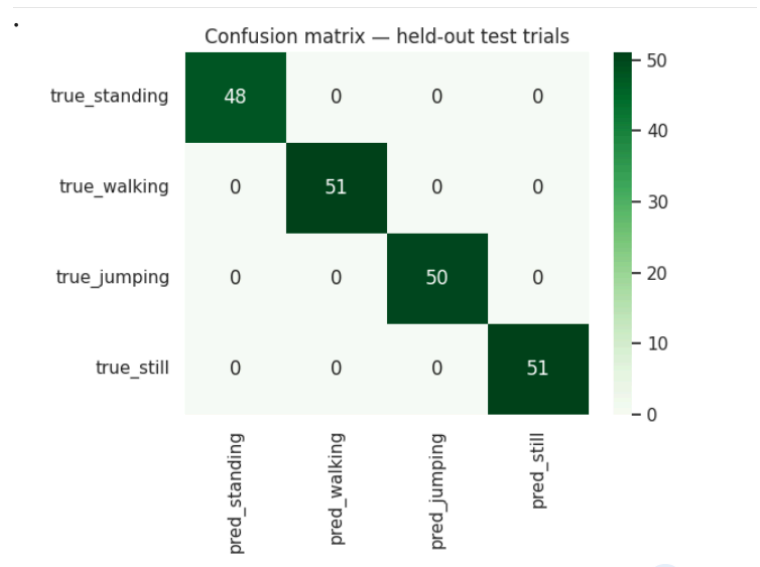


Fig. 7. Confusion matrix heatmap on held-out trials

TABLE VI
CONFUSION MATRIX COUNTS

True\Pred	standing	walking	jumping	still
standing	48	0	0	0
walking	0	51	0	0
jumping	0	0	50	0
still	0	0	0	51

This means that the Hidden Markov Model successfully distinguished the 4 activities.

5.3 Per-class metrics

TABLE VII
SENSITIVITY, SPECIFICITY,& ACCURACY PER ACTIVITY

Activity	No. of samples	Sensitivity	Specificity	Overall Accuracy
standing	48	1.0	1.0	1.0
walking	51	1.0	1.0	1.0
jumping	50	1.0	1.0	1.0
still	51	1.0	1.0	1.0

As shown on Table VII above, all 4 activities had an overall accuracy of 1.0 meaning that the model had perfect recognition on the test set with unseen trials.

5.4 Robustness check

Despite the test split achieving perfect classification, I conducted 4 different trial combinations to evaluate the robustness of the model.

TABLE VIII
ACCURACY ACROSS ALL 4 DIFFERENT HELD-OUT SPLITS

Held-out trials	Test accuracy
(1,2,3)	1.000
(4,5,6)	0.775
(7,8,9)	0.759
(11,12,13)	1.000

Table VIII above shows that accuracy varied between 0.759 to 1.000 depending on the trials withheld for testing. (1,2,3) and (11,12,13) splits had perfect performance but (4,5,6) and (7,8,9) splits achieved accuracies of 0.775 and 0.759 respectively. This means that the model performs well overall, but the performance depends on the characteristics of unseen trials.

6. Discussion and Reflection

According to the results, still was the easiest activity for the Hidden Markov Model to recognise. This is because it produced a stable accelerometer and gyroscope signals because of minimal

activity compared to walking, jumping, and standing which were more difficult to distinguish the evaluation splits. Walking, jumping, and standing cause overlap in some feature values due to the body movements increasing the chances of confusion.

Moreover, the transition matrix was almost perfectly diagonal because of the isolated recordings of single activity trials. This is not realistic in the real world because in fitness monitoring applications, users frequently change between activities such as jumping, standing, and walking, and continuous multi-activity recordings would be needed to capture those changes.

In addition, the 100HZ I used for this was efficient for the gait and jump cycles. The real noise observed was the window-boundary effects at the beginning and end of activity recordings and not the sampling rate itself.

In future, I could enhance the robustness of the activity recognition system by using continuous multi-activity recordings which would allow the HMM to learn transitions and improve temporal modelling. I would also incorporate orientation-independent magnitude-based features to reduce sensitivity to smartphone placement. Finally, I would also increase the number of trials per activity in order to improve parameter estimation.

7. Conclusion

This project successfully classified standing, walking, jumping, and still activities from accelerometer and gyroscope data by combining time and frequency domain feature extraction with a Hidden Markov Model. The pipeline was able to model the temporal nature of human movement and achieved test accuracies ranging from 75.9% to 100%. The main limitation was that I used single activity trials instead of continuous multi-activity recordings. The model was not able to transition properly between activities that is why in future I plan to implement the continuous multi-activity recordings which would be more realistic to use in the real world.

8. References

- [1] A. Bulling, U. Blanke, and B. Schiele, "A tutorial on human activity recognition using body-worn inertial sensors," *ACM Comput. Surv.*, vol. 46, no. 3, pp. 1-33, Jan. 2014.
- [2] L. R. Rabiner, "A tutorial on hidden Markov models and selected applications in speech recognition," *Proc. IEEE*, vol. 77, no. 2, pp. 257-286, Feb. 1989.

- [3] Sensor Logger, "Sensor Logger mobile application," 2024. [Online]. Available: <https://www.tszheichoi.com/sensorlogger>
- [4] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825-2830, 2011.
- [5] L. E. Baum, T. Petrie, G. Soules, and N. Weiss, "A maximization technique occurring in the statistical analysis of probabilistic functions of Markov chains," Ann. Math. Statist., vol. 41, no. 1, pp. 164-171, 1970.
- [6] A. J. Viterbi, "Error bounds for convolutional codes and an asymptotically optimum decoding algorithm," IEEE Trans. Inf. Theory, vol. 13, no. 2, pp. 260-269, Apr. 1967.